

Zomato Rating Prediction

Data Cleaning & Feature Engineering — Interview Prep Notes

This document covers, for every feature in the project: what encoding was used, why it was chosen over the alternatives, the real tradeoffs involved, and likely follow-up questions an interviewer might ask — plus a broader set of data cleaning and feature engineering questions to be ready for.

Section 1 — Encoding Decisions, Feature by Feature

1. `online_order`, `book_table`

Encoding used: Simple binary encoding (Yes/No → 1/0).

Why this and not an alternative: Only two possible values exist — there is no cardinality problem, no ordering issue, and no leakage risk. Binary encoding is the simplest correct choice; any other encoding (one-hot, frequency, target) would be unnecessary complexity for a feature this simple.

Tradeoffs: None meaningful — this is close to a decision with no real downside.

Likely follow-ups:

- Why not one-hot encode this instead of binary? (Answer: for a 2-category variable, one-hot would create a redundant second column that's just the inverse of the first — binary is mathematically equivalent and simpler.)
- Does the order of 1/0 matter here? (Answer: no, since the model doesn't assume ordinality for a single binary column — it just learns a coefficient/split either way.)

2. `listed_in(type)`

Encoding used: One-hot encoding (with `drop_first=True`).

Why this and not an alternative: Low cardinality (roughly 6-7 unique categories: Delivery, Dine-out, Buffet, Cafes, Pubs and Bars, Desserts, etc.) makes one-hot practical without creating excessive sparse columns. Since one-hot is feasible, it's the safest default — no leakage risk, no reliance on the target variable, no noisy small-sample averages.

Tradeoffs: One-hot encoding assumes categories are independent and unordered, which is correct here. The only real cost is a handful of extra columns, which is negligible at this cardinality. `drop_first=True` avoids the 'dummy variable trap' (perfect multicollinearity), at the small cost of making one category the implicit baseline.

Likely follow-ups:

- Why one-hot here but frequency encoding for `location/rest_type`? (Answer: cardinality — one-hot is only practical when the category count is small; `location/rest_type` have far more unique values.)
- What does `drop_first=True` actually do, and why use it? (Answer: drops one dummy column since it's redundant — if you know a row is 'not Dine-out' and 'not Buffet', you already know it must be the third category. Prevents multicollinearity in linear models.)
- Is '`listed_in(type)`' really a quality signal, or does it reflect business model/logistics rather than food quality? (Good self-critique: e.g., buffet-style or delivery-only listings may rate differently for structural reasons — inconsistent buffet quality, cold delivery food — not necessarily reflecting the restaurant's true quality.)

3. location

Encoding used: Frequency encoding — each location replaced with how often it appears in the dataset (restaurant density in that area).

Why this and not an alternative: High cardinality (50+ unique localities) rules out one-hot encoding, which would create dozens of mostly-empty sparse columns. Frequency encoding compresses this into a single informative-ish numeric column without touching the target variable, so it carries zero leakage risk.

Tradeoffs / honest limitation: Frequency encoding measures restaurant **density**, not restaurant **quality** — a location having many restaurants doesn't inherently mean those restaurants rate higher or lower. It's a prevalence proxy, and any correlation it shows with rating in the SHAP plot is likely coincidental (e.g., a commercial hub happening to have decent average ratings) rather than causal.

Likely follow-ups:

- Does a location having many restaurants actually mean it rates well? (Answer: no — this is a genuine limitation worth naming upfront. Restaurant density could reflect oversaturation just as easily as it reflects a thriving food scene.)
- What would be a more direct way to encode location? (Answer: target encoding — average rating per locality — directly answers 'do restaurants here tend to rate well', at the cost of leakage risk that needs K-Fold handling plus smoothing for low-sample localities.)
- Why not just one-hot encode the top 10-15 locations like you did for cuisines? (Reasonable alternative — worth acknowledging as an option, tradeoff being some information loss for the 'other' bucket vs. frequency encoding retaining a continuous signal for every location.)

4. rest_type

Encoding used: Frequency encoding, same logic as location.

Why this and not an alternative: Medium-high cardinality (~20-30 unique types) makes one-hot impractical for the same sparsity reasons as location. Frequency encoding was chosen over target encoding to avoid leakage/small-sample noise risk.

Tradeoffs / honest limitation: Same structural issue as location — 'how common is this restaurant type' is not the same thing as 'does this restaurant type rate well'. Two equally common restaurant types (say, both at 40% frequency) could have completely different average ratings, and frequency encoding would represent them identically, hiding that difference from the model.

Likely follow-ups:

- If two categories have the same frequency but very different average ratings, does the model see them as identical? (Answer: yes, under pure frequency encoding — this is exactly why target encoding is a more direct alternative, since it would give them very different values.)
- Would you consider target encoding here in a real production version? (Good opportunity to walk through K-Fold target encoding with smoothing as the 'correct' upgrade path, and why it wasn't used in this version — complexity vs. payoff tradeoff for a portfolio-scale project.)

5. cuisines

Encoding used: Multi-label binary encoding — one binary column per top-15 most common cuisines, plus a separate `cuisine_count` numeric feature.

Why this and not an alternative: cuisines is fundamentally different from the other categorical columns because it's **multi-label** — a single restaurant can serve several cuisines at once (e.g., 'North Indian, Chinese, Biryani'), so a

normal one-hot or frequency encode (which assume one category per row) don't directly apply. Capping at the top 15 avoids creating 90-100+ sparse columns that full multi-hot encoding of every unique cuisine would create.

Tradeoffs: Rare cuisines outside the top 15 are dropped entirely from binary representation (though `cuisine_count` still reflects their presence in the count). This trades some information loss for a much smaller, less sparse feature set.

Alternatives considered and why not used:

- **Full multi-hot (all cuisines):** No information loss, but creates 90-100+ mostly-empty columns — sparsity hurts tree-based models and bloats the feature space.
- **TF-IDF:** Same column count as full multi-hot, but weights rare cuisines higher and common ones lower. Conceptually a repurposed NLP technique for structured data — doesn't reduce dimensionality, just reweights it.
- **Target encoding (aggregated across a restaurant's cuisine list):** Would collapse cuisines into a single column and directly encode quality signal, but requires careful K-Fold implementation to avoid leakage, plus a decision on how to aggregate multiple cuisines per restaurant (mean, max, etc.) — the most promising upgrade path, but the most complex to implement correctly.

Likely follow-ups:

- Why top 15 specifically, and not top 10 or top 20? (Honest answer: a reasonable cutoff based on the cuisine frequency distribution from EDA — worth checking your own notebook's cuisine value counts to justify the exact number, e.g. 'the top 15 covered X% of all cuisine mentions'.)
- How would you handle a brand-new restaurant serving a cuisine outside your top 15? (Answer: all its cuisine flags would be 0, but `cuisine_count` would still reflect however many cuisines it serves — some information is preserved, some is lost.)
- Why binary flags instead of TF-IDF or target encoding? (Simplicity and zero leakage risk, at the cost of not weighting rare/distinctive cuisines or directly capturing quality signal.)

6. votes — dropped entirely (important design decision)

What happened: votes was initially log-transformed and included (`votes_log`), and was by far the strongest SHAP driver of predicted rating. It was later removed from the feature set entirely.

Why it was dropped: votes is not genuinely available at prediction time for the realistic use case this model is meant to serve — a new or hypothetical restaurant wouldn't have vote history yet. Additionally, votes and rating grow together over time as a restaurant gains traction, making votes closer to a symptom of an already-established rating than an independent predictor of it. Keeping it would make the model appear more accurate than it actually is for real-world prediction scenarios.

Tradeoff of removing it: RMSE/R² got meaningfully worse after dropping votes, since it was the single strongest signal in the dataset. This is an intentional, explainable tradeoff — accuracy on paper vs. genuine real-world usability of the model.

Likely follow-ups:

- Isn't dropping your best feature going to hurt your metrics? Why do it anyway? (This is the key question — be ready to clearly state the leakage-adjacent reasoning: a feature that reflects the target's own history isn't a fair predictor for new/unseen cases.)
- Is this the same kind of issue as target encoding's leakage risk? (Good connective answer: yes, conceptually similar — both involve a feature that's entangled with the outcome. votes is a clearer, more direct case; target encoding is more subtle and fixable with proper K-Fold handling.)

- Would you use votes if the use case were different — e.g., predicting rating changes for existing, already-popular restaurants? (Answer: yes, that would be a legitimate use case where votes is fairly available at prediction time — the decision to drop it is use-case dependent, not a universal rule.)

Section 2 — Quick Reference Table

Feature	Encoding Used	Cardinality	Uses Target	Main Reason
online_order, book_table	Binary (0/1)	2 (very low)	No	Simplest correct choice for a 2-value field
listed_in(type)	One-hot	~6-7 (low)	No	Low cardinality makes one-hot safe and simple
location	Frequency	50+ (high)	No	One-hot impractical; avoids leakage vs. target encoding
rest_type	Frequency	20-30 (medium-high)	No	Same reasoning as location
cuisines	Top-15 binary + count	90-100+, multi-label	No	Multi-label + high cardinality; avoids sparse full one-hot
votes	Dropped entirely	N/A	N/A	Not available at prediction time; entangled with the target

Underlying pattern to remember: the encoding choice is driven mainly by **cardinality** (how many unique values) and secondarily by **leakage risk** (whether the encoding uses the target variable). One-hot is the default when cardinality is low; frequency/target encoding become necessary as cardinality grows; target encoding specifically trades better predictive signal for leakage risk that must be managed with K-Fold + smoothing.

Section 3 — General Data Cleaning Questions

Q: Why did you drop rows instead of imputing missing values for location, cuisines, rest_type, and cost?

A: These are core fields with relatively low missingness, and there's no reliable way to impute a restaurant's location or cuisine — unlike a numeric field, there's no sensible 'average' to fall back on for a categorical identity field. Dropping a small number of rows was safer than introducing fabricated categorical values.

Q: Why did you handle dish_liked differently — a presence flag instead of dropping the column or imputing it?

A: dish_liked was over 50% missing, which is too sparse to use as a genuine feature. But the missingness itself seemed informative (restaurants with more engagement/reviews were more likely to have this filled in), so converting it to a binary has_dish_liked_info flag preserved that signal without relying on a mostly-empty text column.

Q: Why drop columns like url, address, menu_item, phone, reviews_list?

A: These are either unique identifiers with no generalizable signal (url, address, phone), free-text fields that would need separate NLP treatment to be useful (reviews_list, menu_item), or otherwise not relevant to predicting a numeric rating from structured attributes.

Q: How did you handle the messy 'rate' and 'approx_cost' columns?

A: 'rate' came in as strings like '4.1/5' — parsed by splitting on '/' and converting the first part to float. 'approx_cost' came in with comma separators like '1,200' — parsed by stripping commas before converting to float. Rows where rate still couldn't be parsed were dropped, since rate is the target variable and can't be imputed without leaking information.

Q: What would you do differently if you were starting this cleaning process again?

A: Good opportunity for honest reflection — e.g., 'I dropped rows with missing phone numbers early on, which cost some data unnecessarily since phone isn't a useful feature anyway. In hindsight, I'd only drop rows for fields actually used in modeling.'

Q: How do you decide whether to drop a row vs. drop a column when handling missing data?

A: General rule of thumb: if a large fraction of rows are missing a given column, consider dropping or flag-encoding the column (like dish_liked). If only a small fraction of rows are missing essential fields, dropping those specific rows is usually cheaper than losing an entire column's worth of information.

Section 4 — General Feature Engineering Questions

Q: Why log-transform votes instead of using it raw (in the earlier version before it was dropped)?

A: votes was heavily right-skewed — most restaurants have relatively few votes, while a handful of very popular ones have thousands. Feeding that raw into a linear model lets extreme outliers dominate; $\log_{1p}(\text{votes})$ compresses the range so the model treats differences more proportionally. The '1+' in $\log_{1p}$ avoids $\log(0)$ errors for restaurants with zero votes.

Q: What's the general difference between one-hot encoding, frequency encoding, and target encoding — when would you choose each?

A: One-hot: best for low-cardinality, unordered categories, zero leakage risk. Frequency encoding: useful for high-cardinality categories where one-hot would be too sparse, still zero leakage risk, but only captures how common a category is, not how it relates to the target. Target encoding: captures the target-relationship directly, works for any cardinality, but carries leakage risk that must be managed with K-Fold encoding and smoothing for small-sample categories.

Q: What is data leakage, in your own words, and where in this project were you most worried about it?

A: Data leakage is when information that wouldn't actually be available at prediction time (or that's derived from the target itself) sneaks into the features, making the model look more accurate than it really is. The clearest case in this project was votes — dropped because it's entangled with the target variable's own history. Target encoding, if implemented, would be the next area of concern, requiring K-Fold to avoid the same problem.

Q: Why use RMSE and R^2 specifically to evaluate the models — what do they each tell you that the other doesn't?

A: RMSE gives an absolute error measure in the same units as the target (average rating points off), which is intuitive to communicate. R^2 gives a relative measure — the proportion of variance in rating explained by the model — which is useful for judging how much signal exists in the features at all, independent of scale.

Q: Why compare Linear Regression, Random Forest, and XGBoost specifically?

A: Linear Regression serves as an interpretable baseline that assumes simple linear relationships. Random Forest and XGBoost are both tree-based and can capture non-linear relationships and feature interactions that linear regression can't — XGBoost specifically builds trees sequentially to correct prior errors (gradient boosting), which often performs best on structured/tabular data like this.

Q: What is SHAP, and why use it instead of sklearn's built-in feature_importances_?

A: SHAP (SHapley Additive exPlanations) explains individual predictions, not just overall feature rankings — it shows both the direction (does a high value push rating up or down) and magnitude of each feature's effect, for every single row, and is model-agnostic. sklearn's feature_importances_ only gives a single global ranking, doesn't show direction, is biased toward high-cardinality/continuous features due to how tree splits are computed, and only works for tree-based models.

Q: If you had more time, what would you add to this feature set?

A: K-Fold target encoding for location and rest_type (with smoothing for low-sample categories), properly validated against the current frequency-encoded version by comparing RMSE/ R^2 head to head — turning 'I considered it' into 'I tested it and here's what changed.'

Q: Which of your features do you think are the most conceptually weak, and why?

A: `location_freq` and `rest_type_freq` — both measure category prevalence/density rather than genuine quality signal, so any predictive value they show is likely coincidental correlation rather than a causal relationship with rating. This is a good moment to show self-awareness rather than overselling every feature as equally meaningful.

Tip: the strongest answers in this whole section aren't the ones that defend every choice as perfect — they're the ones that name the real tradeoff honestly and explain what a more rigorous version would look like. That's the pattern used throughout this document.